

目录

GC 优化架构文档 1

概述 1

问题分析 1

 原始数据流（非零拷贝路径） 1

 GC 热点识别 1

优化架构 2

 1. ArrayPool 池化策略 2

 2. 对象复用策略 3

 3. 分片元数据结构 3

 4. 增量 GC 辅助 4

内存流转图 5

配置参数 6

 Unity 项目设置 6

 运行时参数 6

优化效果 6

 预期改进 6

 监控指标 6

注意事项 6

文件变更清单 7

GC 优化架构文档

概述

本文档记录了 RoboMasterClientWMJ 项目视频管线的 GC（垃圾回收）优化架构。由于使用 Intel 集成显卡时无法使用 NVDEC 零拷贝路径，视频数据需要经过多次内存拷贝，导致大量托管堆分配和 GC 压力，引发画面卡顿和帧回退现象。

问题分析

原始数据流（非零拷贝路径）

UDP数据包 → 组帧 → FFmpeg进程(VAAPI解码) → pipe stdout → C#读取byte[] → Texture2D → GPU

GC 热点识别

热点位置	分配频率	单次分配大小	影响
UdpVideoHandler NALU	~30,000次/秒	1-2KB	高频小对象分配
AnnexBFrameAssembler List	~30次/秒	数十字节	频繁临时对象
AnnexBFrameAssembler MemoryStream	~30次/秒	200-300KB	中等大小分配

热点位置	分配频率	单次分配大小	影响
FfmpegPipeDecoder 帧像素	~30次/秒	~11MB (2560×1440×3)	大对象分配

优化架构

1. ArrayPool 池化策略

1.1 NALU 分片池化 (UdpVideoHandler)

```
// 使用共享ArrayPool
private static readonly ArrayPool<byte> naluPool = ArrayPool<byte>.Shared;

// 租用缓冲区
byte[] nalu = naluPool.Rent(naluLen);
span.Slice(8).CopyTo(nalu);

// UdpVideoFrame 支持池化
public class UdpVideoFrame
{
    public byte[] Nalu;
    public int NaluActualLength; // 实际数据长度 (Rent可能返回更大数组)
    public bool IsPooled;       // 是否来自池

    public void ReturnNaluToPool()
    {
        if (IsPooled && Nalu != null)
        {
            ArrayPool<byte>.Shared.Return(Nalu);
            Nalu = null;
            IsPooled = false;
        }
    }
}
```

1.2 解码帧像素池化 (FfmpegPipeDecoder)

```
var pool = System.Buffers.ArrayPool<byte>.Shared;

// 租用像素缓冲区
byte[] framePixels = pool.Rent(dataSize);
Buffer.BlockCopy(readBuffer, 0, framePixels, 0, dataSize);

// DecodedFrame 支持池化
public class DecodedFrame
{
    public byte[] Pixels;
```

```

public int PixelArraySize; // 实际使用字节数
public bool IsPooled;

public void ReturnToPool()
{
    if (IsPooled && Pixels != null)
    {
        ArrayPool<byte>.Shared.Return(Pixels);
        Pixels = null;
        IsPooled = false;
    }
}
}

```

2. 对象复用策略

2.1 预分配 List 复用 (AnnexBFrameAssembler)

```

// 预分配, 避免每次方法调用分配新List
private readonly List<ushort> tempRemoveList = new List<ushort>(32);

// 使用时清空复用
tempRemoveList.Clear();
foreach (var kv in buffers)
{
    if (shouldRemove) tempRemoveList.Add(kv.Key);
}

```

2.2 MemoryStream 复用 (AnnexBFrameAssembler)

```

// 预分配大容量MemoryStream
private readonly MemoryStream assembleStream = new MemoryStream(512 * 1024);

// 组帧时复用
assembleStream.SetLength(0);
assembleStream.Position = 0;
foreach (var slice in slices)
{
    assembleStream.Write(slice.Data, 0, slice.ActualLength);
}
annexB = assembleStream.ToArray(); // 仍需分配输出 (解码器需独立副本)

```

3. 分片元数据结构

为支持正确的池化归还, 引入 SliceData 结构:

```

private struct SliceData
{
    public byte[] Data; // 数据缓冲区
}

```

```

    public int ActualLength;    // 实际数据长度
    public bool IsPooled;      // 是否来自池
}

// FrameBuffer 使用 SliceData
public SortedDictionary<ushort, SliceData> Slices;

// 帧清理时归还所有分片
public void ReturnSlicesToPool()
{
    foreach (var kv in Slices)
    {
        if (kv.Value.IsPooled && kv.Value.Data != null)
        {
            ArrayPool<byte>.Shared.Return(kv.Value.Data);
        }
    }
    Slices.Clear();
}

```

4. 增量 GC 辅助

4.1 定期增量 GC 提示

```

private int framesSinceLastGcHint;
private const int GC_HINT_INTERVAL_FRAMES = 60; // 每60帧

// Update() 末尾
framesSinceLastGcHint++;
if (framesSinceLastGcHint >= GC_HINT_INTERVAL_FRAMES)
{
    framesSinceLastGcHint = 0;
    if (GarbageCollector.isIncremental)
    {
        // 分配1ms时间片给增量GC
        GarbageCollector.CollectIncremental(1000000);
    }
}

```

4.2 空闲时低优先级 GC

```

private float lastFullGcTime;
private const float FULL_GC_INTERVAL_SEC = 10f;

// 仅在没有解码积压时触发
if (nowGc - lastFullGcTime >= FULL_GC_INTERVAL_SEC)
{
    if (decodedSinceLastApply <= 2) // 空闲判定

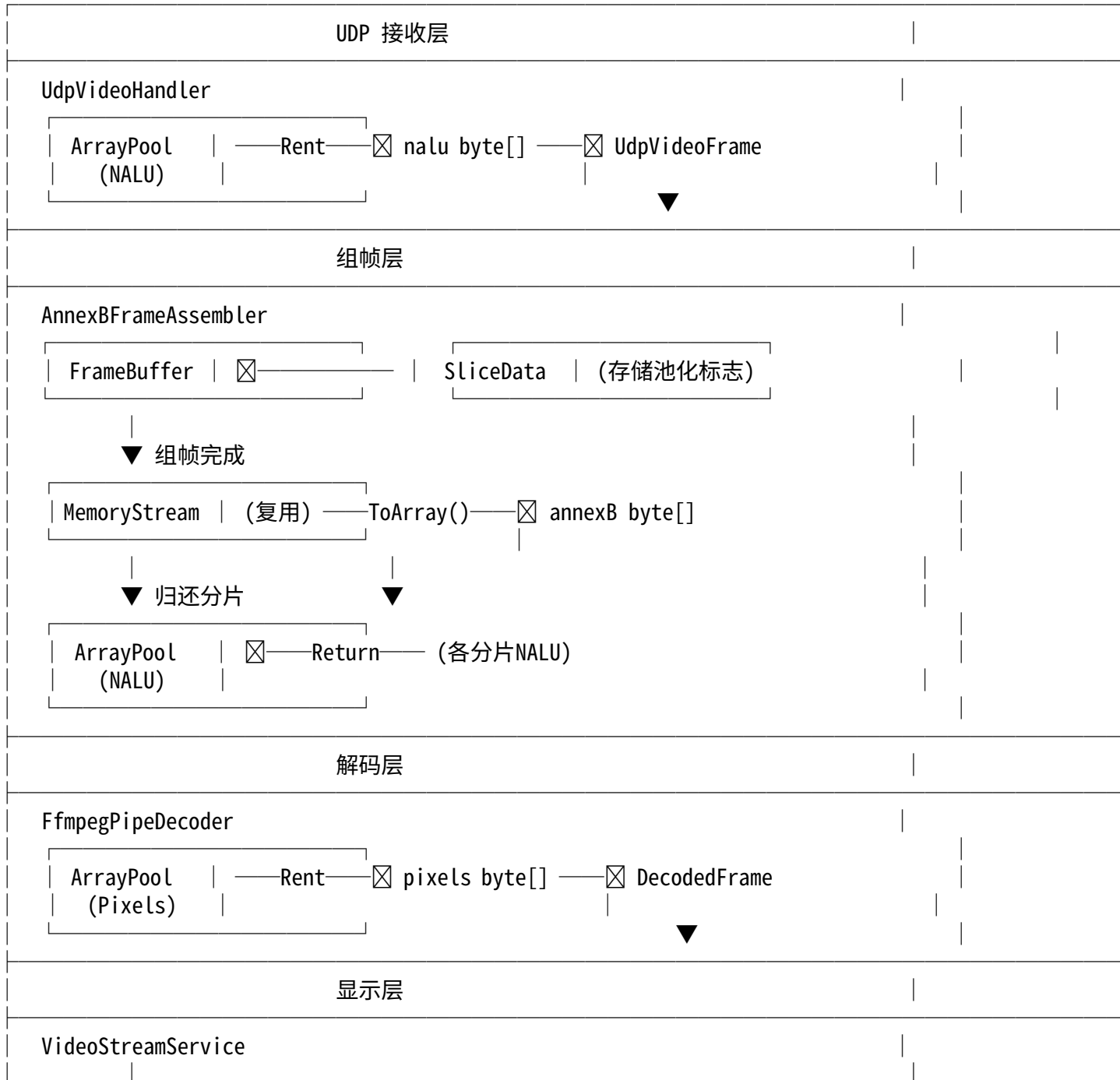
```

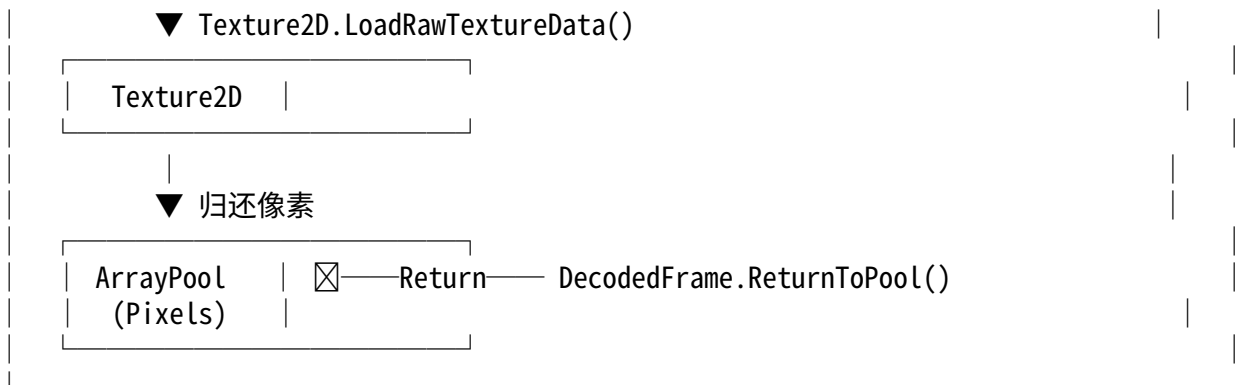
```

{
    lastFullGcTime = nowGc;
    // 仅回收Gen 0, 开销最小
    System.GC.Collect(0, GCCollectionMode.Optimized, false);
}

```

内存流转图





配置参数

Unity 项目设置

```
# ProjectSettings/ProjectSettings.asset
gcIncremental: 1 # 启用增量GC
```

运行时参数

参数	默认值	说明
GC_HINT_INTERVAL_FRAMES	60	增量GC提示间隔帧数
FULL_GC_INTERVAL_SEC	10.0	低优先级完整GC间隔秒数
maxDrainPerUpdate	16	每帧最大消费解码帧数
maxQueueSize	8	解码队列最大长度

优化效果

预期改进

1. **减少 GC 频率**：大幅减少每秒托管堆分配次数
2. **平滑 GC 停顿**：增量 GC 将回收工作分散到多帧
3. **降低内存碎片**：ArrayPool 缓存常用大小数组
4. **避免 LOH 压力**：大对象复用减少大对象堆分配

监控指标

使用 Unity Profiler 观察：- **GC.Alloc**：每帧分配量应显著降低 - **GC.Collect**：应看到更多小的增量回收
- **Managed Heap Size**：应趋于稳定而非持续增长

注意事项

1. **归还顺序**：确保在数据使用完成后再归还到池
2. **ActualLength**：使用 ActualLength 而非 Array.Length（Rent 可能返回更大数组）
3. **IsPooled 标志**：非池化数组不应调用 Return
4. **线程安全**：ArrayPool<T>.Shared 是线程安全的

文件变更清单

文件	变更内容
UdpVideoHandler.cs	NALU ArrayPool 池化
AnnexBFrameAssembler.cs	SliceData 结构、List/MemoryStream 复用、NALU 归还
FfmpegPipeDecoder.cs	像素 ArrayPool 池化
IVideoDecoder.cs	DecodedFrame 池化支持
VideoStreamService.cs	增量 GC 辅助、帧归还

文档创建日期：2026年2月9日